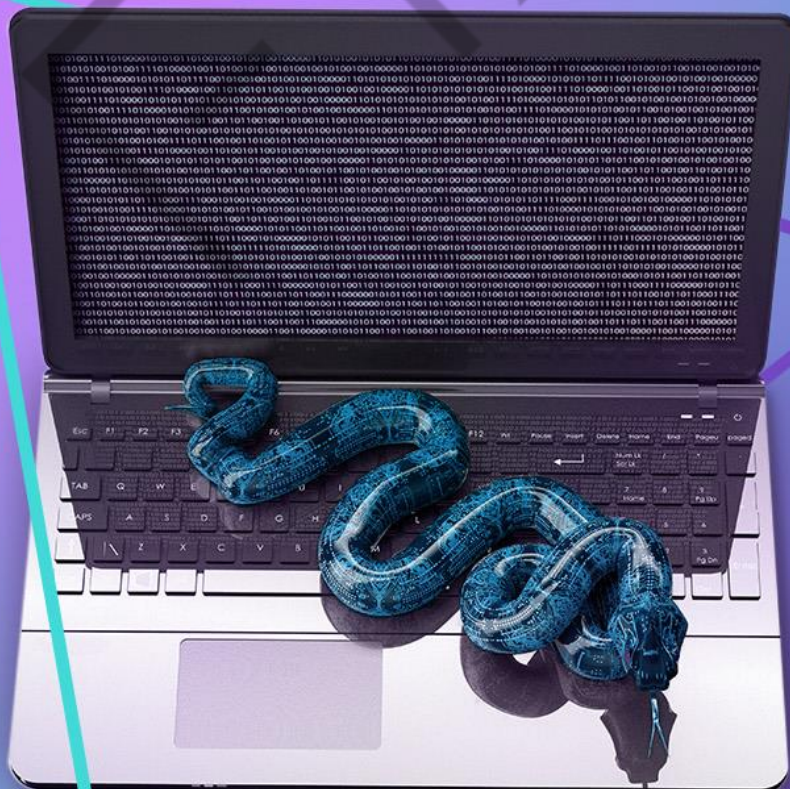


PYTHON O CÓDIGO DA
RESISTÊNCIA CONTRA O IMPÉRIO

PYTHON: FUNÇÃO SEGURA



8

LISTA DE FIGURAS

Figura 1 – Servidor.....	8
Figura 2 – Cliente	10
Figura 3 – Executando o servidor.....	11
Figura 4 – Executando o cliente	12
Figura 5 – Resposta do servidor	12

EXEMPLO

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Retornando IP	6
Código-fonte 2 – Identificando portas.....	7
Código-fonte 3 – Código para servidor.....	8
Código-fonte 4 – Código para cliente	10
Código-fonte 5 – Código para servidor chat	13
Código-fonte 6 – Código para cliente chat	13
Código-fonte 7 – Código para finalizar o cliente.....	14
Código-fonte 8 – Código para servidor UDP	15
Código-fonte 9 – Código para cliente UDP	17
Código-fonte 10 – Código para teste de porta aberta	18
Código-fonte 11 – Código para Port Scan.....	19

SUMÁRIO

1 ÁREAS DE ATUAÇÃO EM PYTHON.....	5
2 TUDO COMEÇA COM A BIBLIOTECA <i>SOCKET</i>	5
2.1 Preparação do servidor	7
2.2 Preparação do cliente	10
2.3 Chat Python.....	12
2.4 Protocolo UDP.....	15
3 O CÓDIGO-FONTE FAZ CONTATO COM O SERVIDOR.....	17
3.1 O mundo não é feito só de chats.....	17
REFERÊNCIAS.....	21

1 ÁREAS DE ATUAÇÃO EM PYTHON

Neste conteúdo, veremos como o Python pode se interligar com diversas áreas. Até o momento, no seu curso, foram apresentados os conceitos básicos da linguagem e um pouco de aplicações externas, como, por exemplo, a utilização dos pacotes externos. A missão, a partir deste capítulo, é demonstrar as várias áreas de atuação em que poderemos utilizar Python.

Agora, vamos entender como ele pode interagir com alguns protocolos, e o resultado dessa interação nos permitirá criar códigos-fontes ainda mais completos e realizar os primeiros testes de cybersecurity. Então, abra o seu IDE e vamos “codar”.

2 TUDO COMEÇA COM A BIBLIOTECA SOCKET

A biblioteca *socket* integra o conjunto de bibliotecas-padrão que já fazem parte da PVM do Python, ela não precisa ser instalada por meio do comando “pip”, apresentado anteriormente. Para utilizar essa biblioteca, basta realizar o import na primeira linha do seu código-fonte, da seguinte forma:

```
import socket
```

Você deve estar se perguntando: por que preciso importar essa biblioteca? Qual é a sua finalidade? Pois bem, essa é a biblioteca que será responsável por criar ferramentas e estabelecer comunicação entre dois processos ou mais, que podem estar em uma rede de computadores. Por isso, vamos relembrar os principais protocolos envolvidos em uma transmissão de dados entre computadores:

- **IP:** representa o endereço do qual partiu ou para o qual seguirá determinado pacote de dados, ou seja, o endereçamento lógico;
- **TCP:** protocolo de transporte que possui maior confiabilidade na entrega dos pacotes, devido ao processo de *handshake* que ele realiza; e
- **UDP:** protocolo de transporte que possui maior velocidade, porém menor confiabilidade de entrega (quando comparado ao TCP), uma vez que ele não realiza *handshake*.

Claro que essas definições pretendem apenas relembrar os conceitos básicos de tais protocolos.

Vejamos um primeiro exemplo para recuperarmos o endereço IP por meio do DNS:

- Crie um novo diretório chamado: **Capitulo11_Socket**, dentro dele, crie um arquivo do tipo “Python File”, chamado: **Socket.py**, e digite o seguinte código-fonte:

```
import socket
resp="S"
while(resp=="S"):
    url=input("Digite uma url: ")
    ip=socket.gethostbyname(url)
    print("O IP referente à url informada é: ", ip)
    resp=input("Digite <s> para continuar: ").upper()
```

Código-fonte 1 – Retornando IP
Fonte: Elaborado pelo autor (2020)

Nesse exemplo, estamos:

- Importando a biblioteca *socket*.
- Criando uma variável para controlar o nosso laço, que virá na sequência.
- Criando um laço para que, enquanto você digita a letra “s”, o código-fonte continue lhe perguntando uma URL e exibindo o seu IP. O laço somente será encerrado caso você digite algo diferente de “s” ou se digitar uma URL inexistente.
- Dentro do laço, temos uma variável chamada “url” que simplesmente armazenará o domínio que está tentando localizar o IP, por isso, digite algo como: <www.globo.com>.
- Logo depois, colocamos a variável “ip” para receber o conteúdo do método *gethostbyname()*, que será o IP baseado no valor da variável “url”, que, por sua vez, está dentro dos parênteses, ou seja, *gethostbyname()* é um método pertencente à classe *socket* que recebe uma URL e retorna o IP.
- Por fim, exibimos o IP na tela para o usuário.

Simple, não é mesmo? Mas, até o momento, vimos o *socket* agindo apenas junto ao protocolo IP, que, por sua vez, ainda necessita de uma porta para que pacotes possam ser direcionados de maneira íntegra e correta.

Podemos identificar as portas da nossa máquina que estão sendo utilizadas para serviços, como domínio, http e ftp, por exemplo. Crie um novo arquivo “Python File” com o nome Portas.py e digite o código-fonte a seguir:

```
import socket

print(socket.getservbyname("domain"))
print(socket.getservbyname("http"))
print(socket.getservbyname("ftp"))
```

Código-fonte 2 – Identificando portas

Fonte: Elaborado pelo autor (2018)

Com a execução desse código-fonte, teremos o retorno das portas que estamos disponibilizando para: domínio (por padrão 53), usado para resolver a conversão entre DNS e IP; HTTP (por padrão 80), usado para navegar nas páginas WEB; e FTP (por padrão 21), usado para a transferência de arquivos.

Existem “n” portas que possuem números padrões definidos pela IANA e que permitem o acesso a serviços diversos. Para mais informações sobre as portas, recomendo consultar o link: <<http://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml>>.

Agora que já sabemos como recuperar um IP e como identificar as portas, já é possível estabelecer um diálogo entre dois processos que podem estar sendo executados em máquinas separadas em uma mesma rede, ou não; os processos podem estar rodando em um mesmo computador.

2.1 Preparação do servidor

Vamos preparar duas ferramentas, uma delas permanecerá no servidor, isso significa que ficará escutando e atendendo aos pedidos dos clientes. A segunda ferramenta deverá ficar nos clientes, ou seja, é a ferramenta que realizará as solicitações ao servidor.



Figura 1 – Servidor
Fonte: www.penso.com.br (2020)

Vamos começar criando um arquivo “Python File”, chamado **Servidor.py**, que a princípio deverá conter os seguintes comandos:

```
from socket import *
servidor="127.0.0.1"
porta=43210
obj_socket = socket(AF_INET, SOCK_STREAM)
obj_socket.bind((servidor,porta))
obj_socket.listen(2)
print("Aguardando cliente....")
while True:
    con, cliente = obj_socket.accept()
    print("Conectado com: ", cliente)
    while True:
        msg_recebida = str(con.recv(1024))
        print("Recebemos: ", msg_recebida)
        msg_enviada = b'Olah cliente'
        con.send(msg_enviada)
        break
    con.close()
```

Código-fonte 3 – Código para servidor
Fonte: Elaborado pelo autor (2020)

Vamos a uma explicação sobre o que está ocorrendo nesse código-fonte:

- Primeiramente, importamos todas (*) as funções da biblioteca *socket*, isso será necessário para que possamos criar objetos do tipo *socket*.
- Criamos duas variáveis, “servidor” e “porta”, para armazenarmos esses dois dados que serão utilizados posteriormente; em “servidor”, poderíamos utilizar também a palavra “localhost” para identificarmos que o servidor é a

própria máquina que está executando o código-fonte, o que também é representado pelo endereço “127.0.0.1” aqui utilizado. Em “porta”, escolhemos uma porta representada por um número inteiro entre 0 e 65535; normalmente as portas entre 0 e 1023 são utilizadas, por padrão, para atribuições de serviços conhecidos por meio do sistema operacional.

- Na quarta linha, criamos o nosso objeto `socket` “`obj_socket`”, por meio da função “`socket()`”, que exige dois parâmetros: o primeiro definirá a família responsável por identificar os pacotes. Para o nosso exemplo, definimos como `AF_INET`, o que significa que vamos utilizar a identificação do emissor/receptor do(s) pacote(s) via DNS ou número IP (poderíamos utilizar a constante `AF_UNIX`, o que mudaria a forma de identificar a origem/destino do[s] pacote[s]). Já o segundo parâmetro refere-se ao transporte desse pacote, que pode ser `SOCK_STREAM`, representando o protocolo TCP (pelo qual nós optamos), ou `SOCK_DGRAM`, representando o protocolo de transporte UDP.
- Na próxima linha, fazemos a associação no nosso objeto `socket` com o nosso servidor e porta.
- Na linha em que utilizamos a função “`listen()`”, estamos definindo o máximo de clientes que o nosso servidor atenderá simultaneamente; para o nosso caso, definimos que serão, no máximo, dois (2) clientes.
- Montamos, na sequência, dois laços infinitos (enquanto for verdadeiro): no primeiro, aguardamos a chamada de um cliente (por meio da função `accept()`), assim que a tivermos, receberemos uma tupla e vamos direcionar a identificação do cliente para a variável “`cliente`” e a identificação da conexão para a variável “`con`”, e, então, exibiremos a identificação do nosso cliente.
- No segundo laço, aguardando uma solicitação que pode ser transmitida em pacotes de 1024 bytes, exibimos a mensagem recebida e geramos uma mensagem para enviar no formato de “*bytes*” (por isso, a mensagem começa com “`b`” e, em seguida, a *string* que se deseja), enviamos por meio do método “`send()`” e encerramos esse segundo laço *while*.
- Por fim, fechamos a conexão e voltamos a aguardar uma nova conexão.

Todos os dados transmitidos via *socket* devem estar no formato *byte*. O *socket* não envia nem recebe dados *string*, por exemplo.

Veremos, agora, o lado do cliente e o que mudaria em nosso código-fonte.

2.2 Preparação do cliente



Figura 2 – Cliente
Fonte: Google Imagens (2018)

Vamos preparar um código-fonte que permita a comunicação de um cliente qualquer, que faça parte da rede, com o nosso servidor. Para isso, crie um arquivo “Python File”, nomeado como **Cliente.py**, e monte o código-fonte a seguir:

```
from socket import *

servidor="127.0.0.1"
porta=43210

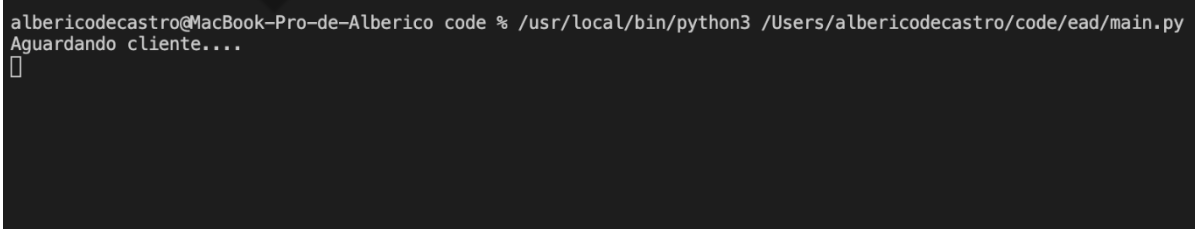
msg = bytes(input("Digite algo: "), 'utf-8')
obj_socket=socket(AF_INET, SOCK_STREAM)
obj_socket.connect((servidor,porta))
obj_socket.send(msg)
resposta=obj_socket.recv(1024)
print("Recebemos: ", resposta)
obj_socket.close()
```

Código-fonte 4 – Código para cliente
Fonte: Elaborado pelo autor (2020)

Detalhando as linhas do código-fonte apresentado:

- Importamos as funções da biblioteca `socket` e criamos duas variáveis, uma para representar o servidor; se o servidor não fosse a máquina local, poderíamos definir o IP do servidor externo, nesse caso. Cuidado ao definir a porta, deve ser a mesma em que inicializamos o servidor.
- Na quarta linha, definimos o conteúdo da variável `"msg"` por meio de uma entrada do usuário (`input`). Observe que utilizamos a função `"bytes()"` para converter o conteúdo do `input`, ou seja, a `string` para o formato bytes, mais uma vez, lembrando: o `socket` transmite somente dados do tipo byte. A função `"bytes()"` possui o segundo parâmetro que faz referência ao padrão de caracteres `"utf-8"`.
- As três próximas linhas são referentes à criação do objeto `socket`, por meio da função `socket()`; à conexão com o servidor, por meio da função `"connect()"`; e, finalmente, enviando uma mensagem para o servidor, utilizando o método `"send()"`.
- A variável resposta recebe o dado enviado pelo servidor, limitando o tamanho para 1024 bytes.
- Finalizamos o código-fonte exibindo a resposta e fechando a conexão.

Agora, surge o momento da execução. Caso você tente executar o `"Cliente.py"`, primeiramente, após digitar o dado a ser transmitido para o servidor, receberá uma mensagem de erro, pois o servidor não está com a porta disponível, uma vez que você não o executou ainda. Dessa forma, execute primeiramente a versão do nosso código-fonte que está no arquivo `"Servidor.py"`; seu console ficará da seguinte forma:



```
albericodecastro@MacBook-Pro-de-Alberico code % /usr/local/bin/python3 /Users/albericodecastro/code/ead/main.py
Aguardando cliente....
█
```

Figura 3 – Executando o servidor
Fonte: Elaborado por Albérico de Castro (2020)

Após colocar o servidor em execução, execute o nosso arquivo `"Cliente.py"`, ele vai solicitar o dado a ser transmitido. Digite o dado que deseja transmitir e receba a resposta do servidor, conforme apresentado na Figura `"Executando o cliente"`:

```
albericodecastro@MacBook-Pro-de-Alberico code % /usr/local/bin/python3 /Users/albericodecastro/code/ead/cliente.py
Digite algo: python
Recebemos: b'0lah cliente'
albericodecastro@MacBook-Pro-de-Alberico code % █
```

Figura 4 – Executando o cliente
Fonte: Elaborado por Albérico de Castro (2020)

De acordo com o destaque da imagem, você poderá ver o que chegou ao servidor e como continua o seu *status* clicando no terminal onde o “Servidor” está em execução, conforme apresentado na Figura “Resposta do servidor”:

```
albericodecastro@MacBook-Pro-de-Alberico code % /usr/local/bin/python3 /Users/albericodecastro/code/ead/main.py
Aguardando cliente....
Conectado com: ('127.0.0.1', 54963)
Recebemos: b'python'
█
```

Figura 5 – Resposta do servidor
Fonte: Elaborado por Albérico de Castro (2020)

Repare que você pode seguir executando o cliente e passando mensagens para ele, que receberá sua mensagem e sempre retornará a mensagem “Olah Cliente”. Observe, também, na guia da execução do servidor, que mostra o IP do cliente e a porta, como não definimos uma porta para o cliente, o próprio *socket* gerou e disponibilizou uma porta aleatória.

O desafio agora é: você consegue montar um chat entre o cliente e o servidor? Ou seja, você pode enviar um dado para o servidor, então faça o mesmo com o servidor e deixe o cliente em um laço infinito para que possam conversar. Tente fazer isso antes de seguir para o próximo tópico, no qual encontrará a resposta para o desafio.

2.3 Chat Python

Vamos demonstrar, agora, como ficará o código-fonte do cliente e do servidor, a fim de montarmos o nosso chat. Vamos começar pelo servidor:

```
from socket import *
servidor="127.0.0.1"
```

```
porta=43210
obj_socket = socket(AF_INET, SOCK_STREAM)
obj_socket.bind((servidor,porta))
obj_socket.listen(2)
print("Aguardando cliente....")
while True:
    con, cliente = obj_socket.accept()
    print("Conectado com: ", cliente)
    while True:
        msg_recebida = str(con.recv(1024))
        print("Recebemos: ", str(msg_recebida)[2:-1])
        msg_enviada = bytes(input("Sua resposta: "), 'utf-8')
        con.send(msg_enviada)
        break
    con.close()
```

Código-fonte 5 – Código para servidor chat
Fonte: Elaborado pelo autor (2020)

Em relação ao novo código-fonte do servidor apresentado, alteramos somente o conteúdo do segundo *while*, no qual estamos convertendo a mensagem que recebemos para *string* (`str(msg_recebida)`) e, em seguida, fazendo um *slice* (`[2:-1]`) dela, em que pegamos do segundo caractere até o último da *string*, com isso, quando recebermos, por exemplo: **b'olah servidor'**, será exibido apenas: **olah servidor**. Alteramos também a linha referente ao conteúdo da variável `msg_enviada`, que antes era uma mensagem única e agora é definida por meio do `input()`, no qual o usuário vai digitar o que desejar.

Em relação ao lado do cliente, vejamos como ficou o código-fonte:

```
from socket import *

servidor="127.0.0.1"
porta=43210

while True:
    obj_socket = socket(AF_INET, SOCK_STREAM)
    obj_socket.connect((servidor, porta))
    msg = bytes(input("Sua mensagem: "), 'utf-8')
    obj_socket.send(msg)
    resposta=obj_socket.recv(1024)
    print("Resposta do Servidor: ", str(resposta)[2:-1])
obj_socket.close()
```

Código-fonte 6 – Código para cliente chat
Fonte: Elaborado pelo autor (2020)

Do lado do cliente, foi incluído o laço *while*, para que o cliente possa mandar mais de uma mensagem ao servidor; deixamos de fora do *while* somente a definição do servidor (IP e porta) e o encerramento da conexão. A resposta do servidor também

foi convertida para *string* e foi realizado o *slice* da *string* para considerar somente a mensagem válida.

Execute o servidor e, em seguida, o cliente, veja que poderá trocar mensagens entre os dois processos. Se estiver em um ambiente com uma rede de computadores, poderá verificar o funcionamento desses códigos-fontes em equipamentos distintos. Claro que muitas aprimorações podem ser realizadas, como, encerrar a conversa do lado do cliente, para isso, bastaria sair do laço infinito:

```
from socket import *

servidor="127.0.0.1"
porta=43210

while True:
    obj_socket = socket(AF_INET, SOCK_STREAM)
    obj_socket.connect((servidor, porta))
    msg = bytes(input("Sua mensagem: "), 'utf-8')
    obj_socket.send(msg)
    resposta=obj_socket.recv(1024)
    print("Resposta do Servidor: ", str(resposta)[2:-1])
    if str(msg)[2:-1].upper()=="FIM":
        break
obj_socket.close()
```

Código-fonte 7 – Código para finalizar o cliente
Fonte: Elaborado pelo autor (2018)

Adicionamos somente as linhas que estão em vermelho; quando digitarmos “fim”, receberemos a mensagem do servidor e, então, a aplicação do lado do cliente será encerrada. Já do lado do servidor, ele continuará escutando novos clientes.

Assim, poderíamos também pensar em muitas outras melhoras, mas a ideia principal deste capítulo é demonstrar como podemos estabelecer comunicação entre dois equipamentos que estejam conectados em um mesmo ambiente de rede, além, é claro, de apresentar a funcionalidade para transmissão de dados entre computadores. Todos os conceitos apresentados serão extremamente úteis para o desenvolvimento de ferramentas em Python que permitirão controlar e testar informações que estarão trafegando na sua rede.

Como sugestão para estudos futuros, caso você precise rodar uma aplicação cliente-servidor em um ambiente de alta concorrência, em outras palavras, com muitos clientes conectando-se simultaneamente ao servidor, é possível que você tenha que optar por trabalhar com *threads* (processos dentro do sistema operacional). Nesse

caso, você tornaria cada conexão com o cliente um *thread* e, com isso, o gerenciamento passaria a ser escalável e propício para a alta concorrência.

Você poderia ainda utilizar a biblioteca *socketserver* com *threads*, assim suas requisições para o servidor estariam ainda mais sob controle. Não abordaremos esses modos neste capítulo, pois eles fazem parte de uma programação mais avançada, fugindo do escopo do conteúdo de Python deste momento, que é o de apresentar as possibilidades de aplicações do Python em um ambiente de infraestrutura e segurança.

2.4 Protocolo UDP

Como você já viu anteriormente, o protocolo UDP não é confiável em relação à garantia de que os dados cheguem até o destino, mas, como contraponto, sabemos que esse protocolo entrega de maneira muito mais rápida em comparação com o TCP. As aplicações desse protocolo estão mais voltadas para ambientes que utilizem *streaming* de vídeos e/ou áudios, ou até mesmo ambientes de jogos on-line, que têm a velocidade como fator primordial para a jogabilidade.

Veremos um pequeno exemplo utilizando o protocolo UDP para que possamos completar a manipulação dos dois principais protocolos de transporte dentro do Python.

Vamos iniciar elaborando o código-fonte do servidor. Para isso, crie o arquivo do tipo “Python File”, chamado **Servidor_UDP.py**, e monte o seguinte código-fonte:

```
from socket import *
servidor="127.0.0.1"
porta=43210
obj_socket = socket(AF_INET, SOCK_DGRAM)
obj_socket.bind((servidor,porta))
print("Servidor pronto....")
while True:
    dados, origem = obj_socket.recvfrom(65535)
    print("Origem.....: ", origem)
    print("Dados recebidos.: ", dados.decode())
    resposta=input("Digite a resposta: ")
    obj_socket.sendto(resposta.encode(), origem)
obj_socket.close()
```

Código-fonte 8 – Código para servidor UDP
Fonte: Elaborado pelo autor (2020)

Vamos às descrições do código-fonte informado:

- Começamos importando e estabelecendo o endereço do servidor e a porta que será utilizada para a comunicação.
- Na terceira linha, alteramos o segundo parâmetro para DGRAM, o que determina que utilizaremos o protocolo de transporte UDP.
- Realizamos o **bind** do servidor com a porta e o endereço especificados.
- Não realizamos o `listen()`, uma vez que não há necessidade de confiabilidade na troca de dados, por isso já entramos no laço.
- Dentro do laço, aguardamos os dados do cliente, com o tamanho limitado a 65535 (tamanho máximo), prática bem comum, devido à aplicação cotidiana desse protocolo. Perceba que o método agora utilizado é o `recvfrom()`, que não possui ligação com o objeto de conexão.
- Os dois `prints()` a seguir servem para exibir a origem e, em seguida, os dados. Perceba que agora não utilizamos mais a conversão para *string* e o *slice*, em vez disso, usamos a função “`decode()`”, que exibe os dados bytes no formato *string*. Essa forma também poderia ser utilizada junto ao protocolo TCP.
- Recebemos a resposta que o servidor deseja passar e encaminhamos para o cliente por meio do método “`sendto()`”, que é formado por dois parâmetros: o primeiro é a mensagem propriamente dita, na qual, dessa vez, utilizamos o método `encode()` para converter de *string* para byte; e o segundo parâmetro é a origem para a qual desejamos enviar a mensagem.
- Então finalizamos a conexão do *socket*.

Crie, agora, um novo arquivo “Python File”, chamado **Cliente_UDP.py**, e digite o código-fonte a seguir:

```
from socket import *

servidor="127.0.0.1"
porta=43210
obj_socket = socket(AF_INET, SOCK_DGRAM)
obj_socket.connect((servidor, porta))
saida=""
while saida!="X":
    msg = input("Sua mensagem: ")
    obj_socket.sendto(msg.encode(), (servidor,porta))
```



```
dados, origem = obj_socket.recvfrom(65535)
print("Resposta do Servidor: ", dados.decode())
saida=input("Digite <X> para sair: ").upper()
obj_socket.close()
```

Código-fonte 9 – Código para cliente UDP
Fonte: Elaborado pelo autor (2020)

As alterações realizadas nesse código-fonte estão destacadas na cor vermelha. Observe que alteramos o segundo parâmetro da função “socket()” para UDP. O método “sendto()” também foi utilizado, pois, diferentemente do método “send()” utilizado no TCP, o “sendto()” possui dois parâmetros: a mensagem e o destino da mensagem. A captura do servidor também foi alterada para o método “recvfrom()”.

Muitas são as possibilidades a partir do conteúdo apresentado neste capítulo, até mesmo criar um *bot* dentro de um ambiente controlado, no qual o cliente faz uma pergunta, que a aplicação do servidor verifica e, com base em uma análise, envia a resposta. Por exemplo, o cliente pergunta: “Qual é o e-mail do administrador?”, “Qual é o ramal do departamento de pessoal?”, “Quem ainda não efetuou login hoje?”.

O *bot* pode fazer uma análise das palavras recebidas, realizar algum processamento e responder de acordo com a análise/processamento dos dados que foram recebidos, ajudando, de maneira automática, todos os clientes que estiverem na rede. Essa ideia pode ser mais aprimorada e, por exemplo, fazer com que o servidor esteja conectado a um banco de dados que armazenará perguntas prováveis e respostas já determinadas, ou ainda estar conectado a alguma inteligência artificial, como Watson da IBM, por exemplo, direcionando as respostas para aqueles que fizerem parte da sua rede.

3 O CÓDIGO-FONTE FAZ CONTATO COM O SERVIDOR

3.1 O mundo não é feito só de chats

A biblioteca *sockets* nos permite interagir com as portas dos servidores por meio dos protocolos de rede, essa característica, que nos permitiu desenvolver uma aplicação simples de chat, pode ser utilizada para desenvolver outras aplicações que nos ajudam no cenário de *cybersecurity*. Um exemplo disso seria uma aplicação de

Port Scan em que o código-fonte faz contato com o servidor e realiza testes abrindo conexões com várias portas a fim de estabelecer quais delas possuem serviços ativos.

```
import socket

saida=""
while saida!="X":
    host = input("Entre o dominio ou ip do servidor: ")
    port = int(input("Entre a porta do servidor: "))

    obj_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    if obj_socket.connect_ex((host,port)):
        print("Port ",port,"Is Closed.")
    else:
        print("Port ",port,"Is Open.")
    saida=input("Digite <X> para sair: ").upper()
```

Código-fonte 10 – Código para teste de porta aberta
Fonte: Elaborado pelo autor (2020)

O código-fonte apresentado acima faz o teste para saber se uma porta específica de um servidor está aberta. Para isso, alguns conceitos que nós já usamos no desenvolvimento da aplicação de chat se repetem nesse código-fonte. O primeiro é a utilização de host e porta a fim de identificar quais servidor e serviço serão testados.

O código-fonte, então, abre uma nova conexão *socket* utilizando o protocolo TCP com o host informado. Diferentemente do chat onde nós utilizávamos um comando “send” para enviar o conteúdo da nossa mensagem para o cliente, aqui é utilizada a função “connect”, a fim de descobrir se eu consigo ou não estabelecer uma conexão com o serviço na porta selecionada.

Ao usar a função “socket.connect(endereço)”, o código-fonte inicia uma conexão com um soquete remoto no endereço especificado. Se a conexão for interrompida por um sinal, o método aguardará até que a conexão seja concluída ou aumentará um soquete. Uma função de “timeout” permite criar códigos mais complexos, nos quais, caso o soquete esteja bloqueado, a tentativa de conexão será encerrada ao atingir o tempo limite.

Já a função “socket.connect_ex(endereço)”, retorna um indicador de erro 0 se a operação for bem-sucedida, caso contrário, o valor da variável erro. Dessa forma, nosso código-fonte é escrito para aguardar uma mensagem de erro (representando que a porta está fechada), e caso o erro seja “0”, a porta está aberta. Essa

característica também deixa o código-fonte mais rápido, permitindo um melhor desempenho ao testar mais de uma porta.

Até agora, o código-fonte se limita a checar se uma única porta está aberta, entretanto, em um código de Port Scan precisamos testar um *range* de portas. Dessa forma, podemos usar um laço *for* para conseguir repetir o teste até atingir um número determinado, que seria 65535.

```
import socket

saida=""
while saida!="X":
    host = input("Entre o dominio ou ip do servidor: ")

    for port in range (1, 65535):
        obj_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        obj_socket.settimeout(1)
        if obj_socket.connect_ex((host,port)):
            print("Port ",port,"Is Closed.")
        else:
            print("Port ",port,"Is Open.")
    saida=input("Digite <X> para sair: ").upper()
```

Código-fonte 11 – Código para Port Scan

Fonte: Elaborado pelo autor (2020)

Ao analisarmos o código-fonte “Código para Port Scan”, acima, podemos ver que algumas mudanças foram realizadas. A primeira é que não existe mais um “input” para o número da porta, uma vez que vamos percorrer todo o *range* entre a porta 1 e a porta 65535. A segunda é que o limite máximo da porta está incrementado em 1, isso porque o último valor não vai ser executado.

Por fim, o código também adiciona um *timeout* para fazer com que portas que demoram mais tempo para responder, tipicamente porque estão fechadas ou por bloqueios de *firewall*, não elevem o nosso tempo de processamento, tornando inviável a execução do código-fonte. Assim, sempre que a espera por resposta exceder um segundo, o código-fonte vai entender que aquela conexão não foi bem-sucedida e assim pulará para a próxima porta.

Agora, imagine que esse tipo de teste pode sobrecarregar a capacidade de processamento de alguns servidores, por isso não devemos sair por aí realizando Port Scan em qualquer site. Alguns servidores têm mecanismos no seu *firewall* para coibir esse tipo de teste, impedindo que ele seja realizado de forma repetitiva. O Google, por exemplo, limita o máximo de requisições idênticas que um usuário pode enviar,

então tenhamos em mente que o uso de códigos-fontes ligados à *cybersecurity* deve ser feito com moderação e em ambiente próprio.

EMANIP

REFERÊNCIAS

FORBELLONE, A. L. V. **Lógica de programação**. 3. ed. São Paulo: Prentice Hall, 2005.

KUROSE, J. F. **Redes de computadores e a Internet**: uma abordagem top-down. 6. ed. São Paulo: Pearson Education do Brasil, 2013.

MAXIMIANO, A. C. A. **Empreendedorismo**. São Paulo: Pearson Prentice Hall Brasil, 2012.

PIVA JÚNIOR, D. **Algoritmos e programação de computadores**. São Paulo: Elsevier, 2012.

PUGA, S. **Lógica de programação e estruturas de dados**. São Paulo: Prentice Hall, 2003.

RAMEZ, E.; SHAMKANT B. N. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

RHODES, B. **Programação de redes com Python**. São Paulo: Novatec, 2015.

STALLINGS, W. **Arquitetura e organização de computadores**. 8. ed. São Paulo: Prentice Hall Brasil, 2010.

VISUAL STUDIO CODE. [s.d.]. Disponível em: <<https://code.visualstudio.com>>. Acesso em: 5 mar. 2021.